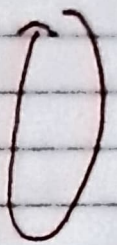


Design and Analysis of Software Systems

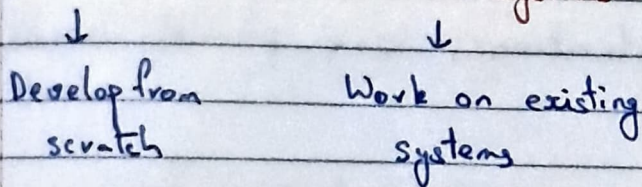
Development Cycle:- Dev Life Cycle [SDLC]

→ Determine whether the proposed sys is viable

- (i) Feasibility Analysis:- Req for proposal
- (ii) Requirements / Specifications:- What is needed, ~~What to~~
- (iii) Design:- How to do
- (iv) Implementation:-
- (v) Testing:- Static and Dynamic & Deployment
- (vi) ~~Main~~ Maintenance:-



Greenfield & Brownfield systems:-



Maintenance is max effort

Modules are never integrated in one shot, multiple steps
During each step, the partially integrated system is tested

Stuff keeps changing, products must evolve

Process Model:- Types of SDLC, SDLC models

- (i) Traditional → Plan driven
- (ii) Agile → Dynamic

Processes must continually improve

De briefing - Planning & Execution

Convergence:- Estimates get more accurate as the rounds

Design and Analysis of Software Systems

Software

Development Cycle:- Dev Life Cycle [SDLC]

→ Determine whether the proposed sys is viable

- Modelling
- (i) Feasibility Analysis:- Req for proposal
 - (ii) Requirements / Specifications:- What is needed, ~~How to do~~
 - (iii) Design:- How to do
 - (iv) Implementation:-
 - (v) Testing:- Static and Dynamic & Deployment
 - (vi) Main Maintenance:-

Greenfield & Brownfield systems:-



Develop from scratch



Work on existing systems

Maintenance is max effort

Modules are never integrated in one shot, multiple steps
 During each step, the partially integrated system is tested

Stuff keeps changing, products must evolve

Process Model:- Types of SDLC, SDLC models

- (i) Traditional → Plan driven
- (ii) Agile → Dynamic

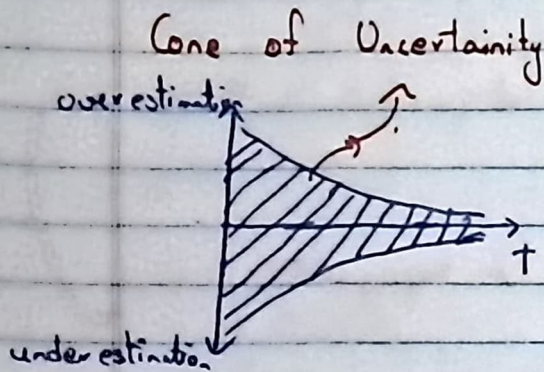
Processes must continually improve

De briefing - Planning & Execution

Convergence:- Estimates get more accurate as the rounds

progressed

Upfront Planning:-



Traditional:- No iterations, one planning session and no changes

Agile:- Many iterations, multiple shorter planning sessions between iterations

Team Dynamics & Self-organization

Leadership and Psychological Safety

Lean Principles

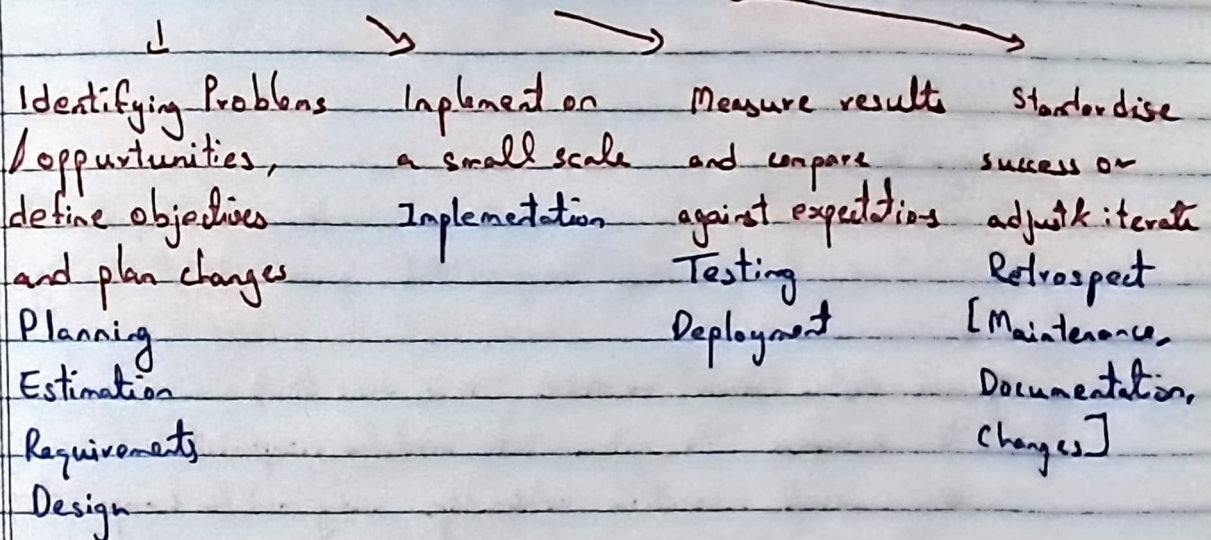
Work in Progress and The Definition of Done
when is the unit $\leftarrow$
actually done

Rework:- Starting from scratch once again
results
→ Technical Debt:-

PDCA - Continuous improvement mindset

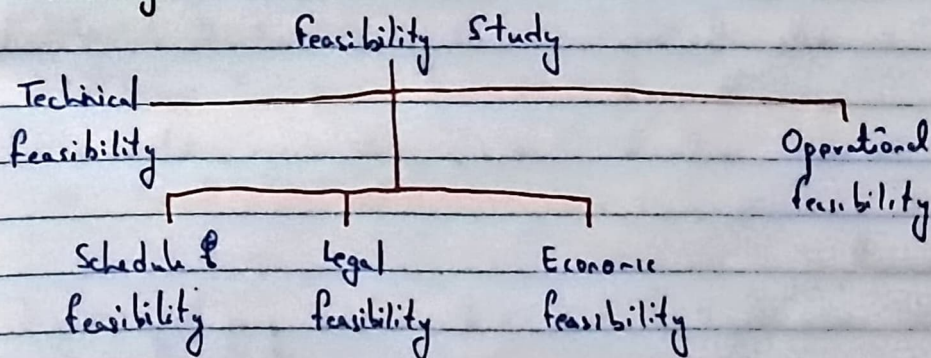
Alexander Deming

Plan - Do - Check - Act



1 PDCA cycle $\approx$ 1 iteration

Feasibility:-



Determine whether the proposed system is viable & worth pursuing

Requirements Engineering:-

Elicit, analyse, specify, validate & manage stakeholder needs so that the system built meets the intended purpose & constraints

SRS - Software Requirement Specifications

PRD - Product Requirement Document

Consists of

- ↳ req gathering & analysis (from diff pous)
- ↳ req specifications
- ↳ verification & validation
- ↳ req management & change control

Design :-

Transform reqs into a robust, maintainable and implementable architecture & detailed designs that guide dev & testing.



High level design (Arch) → ADD (Arch desc doc)

- ↳ decompose the system into modules/components
- ↳ represent invocation relationships among modules/components
- ↳ specify constraints

Detailed Design → DD (Design Doc)

↳ UML

↳ data structures, class diagrams, interfaces & modules are designed

↳ API contracts, seqs, state diagrs, database schemas

Implement :-

- ↳ code front & back end in some prog lang

Testing :-

Test plan ---> Implementation

↓ !---> Design

Test !---> RE

Oracle

Dilbert!
PHD Comics!

7. V-Model

~~Rev~~

High

Maintenance:-

- Prevention → Defective prevention
- Correction → Bug fixes
- Perfection → improvements/enhancements
- Adaptive → Port software to new environments

Process Models:-

Traditional :- (plan driven)

- ↳ Waterfall model
- ↳ ~~Prototyping~~ Prototyping
- ↳ Evolutionary
- ↳ Spiral model et al

Agile :-

- ↳ eXtreme Programming (XP)
- ↳ Scrum
- ↳ Crystal
- ↳ Feature Driven Development (FDD)

Water fall model:- → Stability!

Seq process

Req., design, implementation, testing & maintenance need to be followed in that order

Each phase needs to be completed before moving on.

Challenges:-

- ↳ Heavy weight proc
- ↳ Doc intensive
- ↳ Minimal consumer involvement after req phase
- ↳ Less flexible design
- ↳ Big bang approach to coding & integration
- ↳ One shot delivery opportunity
- ↳ Limited opp for proc improvement

Good for systems with stable requirements

Prototyping Model → Clarity!

Before starting actual dev, working prototypes are built to elicit custome. feedback.

toy model
↓
limited perf & efficiency.

Throwaway Throwaway
↳ Evolutionary.

Good for sys with unclear req, with focus on UI/UX

Morning!

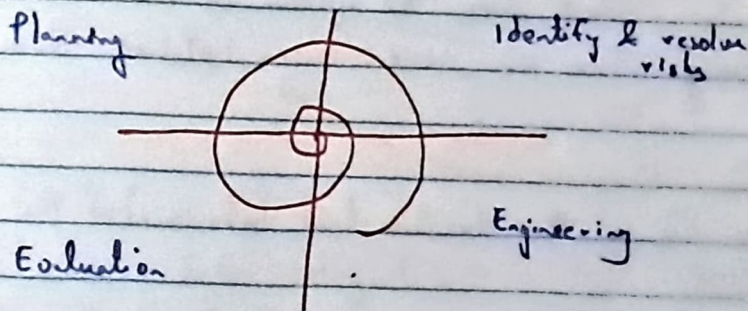
Evolutionary process model $\rightarrow$ Speed!

Sys is broken down into several modules which can be incrementally implemented & delivered.

Iterative incremental model $\rightarrow$ Modern Agile

Spiral Model $\rightarrow$ Safety!

- Strong emphasis on risk.
- Cont. cost feedback
- Sustainable for large, complex high risk systems
- Expensive but highly controlled.



Metaphor model as each loop of the spiral might be a process by itself.

Agile :-

1990 - 2000

- $\rightarrow$ Close collaboration b/w dev & business experts
- $\rightarrow$ 12 practices in the manifesto.
- $\rightarrow$ Maximise face-to-face communication (keep customer in the loop)
- $\rightarrow$ Self organising teams, everyone knows what they are doing and are owners themselves

Chaos.

Many models under Agile

Incremental & Iterative in nature

Characteristics:-

- Short, but many quick releases
- Planning based on user stories
- Each iteration deals with all life cycle activities
- Testing $\Rightarrow$ unit testing for deliverables; acceptance tests for each release
- Flexible Design $\Rightarrow$ evolution vs big upfront effort
- Reflection after each cycle
- Several technical & customer focused presentation opportunities

User Stories:- What does the system do from the user's Φ pov? Many One or more user stories form a feature

Refactoring:- Change the structure (not behaviour) of the code

Dead Code:- Code that is very difficult to test / not possible to test at times

Open-Closed Principle:- Code is open only for extension, closed for modification

M

SOLID principles for OOP

Results in Technical Debt $\rightarrow$ debt incurred cause of bad coding practices.

Code Smell $\leftarrow$ Bad cod (ing) practices)

long methods violate single responsibility principle
↳ Modularize

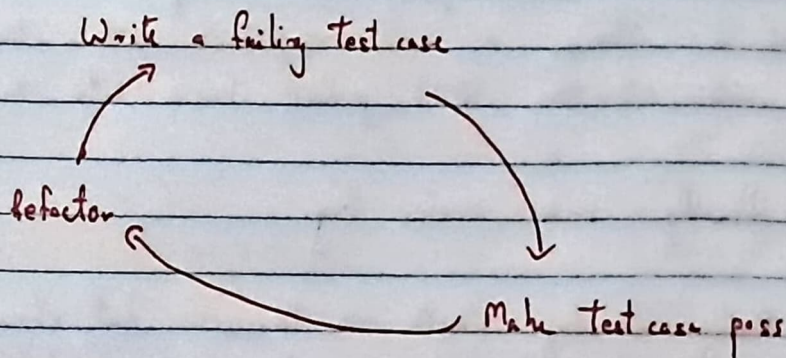
Test Driven Development

Write Tests First!

Test drives the development process

Write and Run Testcases.

Then write code to ensure testcases pass.



Source Code Integration

Big Bang Integration is bad

Source code (download from version control)

Resolve dependencies

Compile & Run

Manual Testing

Package & Deploy.

Continuous Integration $\rightarrow$ DevOps Development practice where devs merge changes into a shared repository frequently.

Commit → Build → Test → Feedback

Automated → Occurs continuously

Build Tools help with build process (MAVEN, ANT)

CD → Continuous Deployment (Automated)

↳ Continuous Delivery (Manual)

SCRUM Process:-

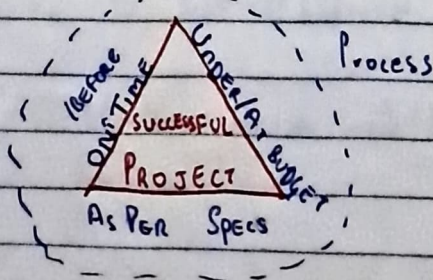
Backlog of User Stories

SCRUM master splits the backlog into multiple sprints.
If a story is not completed within the sprint,
and ~~at the~~ it is pushed back into the backlog.

Usually 2-4 weeks long.

Daily Standup Meets → Discuss what should be done,
what went well, etc to stay in the updated

Project Management



Each need not be estimated.

For estimation

1) Always give a range, never a number

Experience from previous endeavours helps in making more accurate ~~lower~~ estimate -

person hour $\rightarrow$ • work done in 1 hour by 1 person
~~day~~ = 8 person hours
 week = 40 person hours
 year = 2000 person hours

- 2) Always ask what the estimate will be used for.
- 3) Estimation $\neq$ Commitment
- 4) Measure, count and compute (try to atleast). Estimate only when necessary
- 5) Aggregate independent estimates
 $\hookrightarrow$ SLN $\rightarrow$ Planning Power.

Methods of Estimation:-

- i) Top-down
- ii) Bottom-up
- iii) Analogy
- iv) Expert Judgement
- v) Paid to Win
- vi) Parametric / Algorithmic

i) Function Point Analysis -

Software size measured by number & complexity of the fn it performs.

More methodical than LOC counts

Q Count no of fns per category

Q Estimate complexity of fns and apply a weighting multiplier

- ③ Compute on influence multiplier/value adjustment factor and apply
- ④ Results in in point total
- ⑤ Estimating effort & time.

Types of FPA

External Inputs

User/process inputs that creates or updates data.

Data-entry screen, API endpoints etc

External Outputs

Outputs sent outside the application that contain rules.

External Inquiries To

Online queries that retrieves data with little to no processing

Internal Logical Files

Application maintained logical data groups

External Interface Files

Read only logical data used by an application but stored externally

Assign Complexity Weights

For example

story $\approx$ 1 FP

	Input	Output
Low complexity	3 FP	4
Any	4	5
High	6	7

Refer Slides

- Scheduling and Tracking

① Partition your project

Work Breakdown Structure

↓

Checklist of the work that must be accomplished to meet the project objectives

↳ Product / Process / Hybrid WBS

Fin engine

↓

↳ Both Product & Process

Req. Analysis, Design, Testing

Precedence Network

task

Shows the dependency of one task on the other.

↓

Identify the critical path → projects on it can have no slack.

↳ Slack Time $\leftarrow$ Max allowable delay for a non critical activity.

Requirements Engineering

↓

statement that specifies what a system must do, what qualities it must possess to satisfy stakeholder needs

Main reason why projects fail

Traceability:- Very important

Root Cause Analysis for bugs

Is it possible to be absolutely certain of the verity of having correct requirements

Open systems:- Req keep changing (maybe unconstrained)

Closed systems:- (Very) Fixed requirements

We must define the scope by having a more precise problem.

Learn to say No

Scope Creep $\leftarrow$ Scope keeps increasing

Req must
be determined

Clients produced
req

New
Development

A

B

Evolution
of an existing
system

C

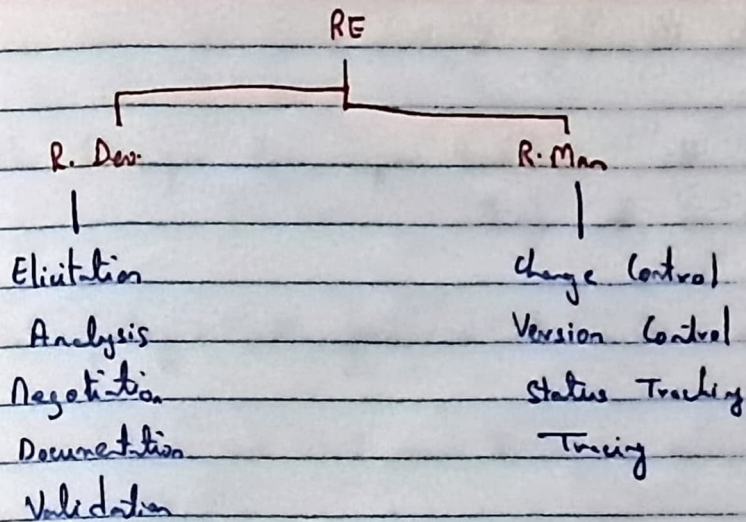
D

$\rightarrow$ Clearly stating reduces scope creep

① Functional Requirements $\leftarrow$ Features / what should the system do (exp)
expressed as functions

② Non-functional Requirements $\leftarrow$ qualities / characteristics of the system which cannot be expressed as fns

$\hookrightarrow$ Clear specific specifications can help measure how well the system is doing what it is supposed to do.



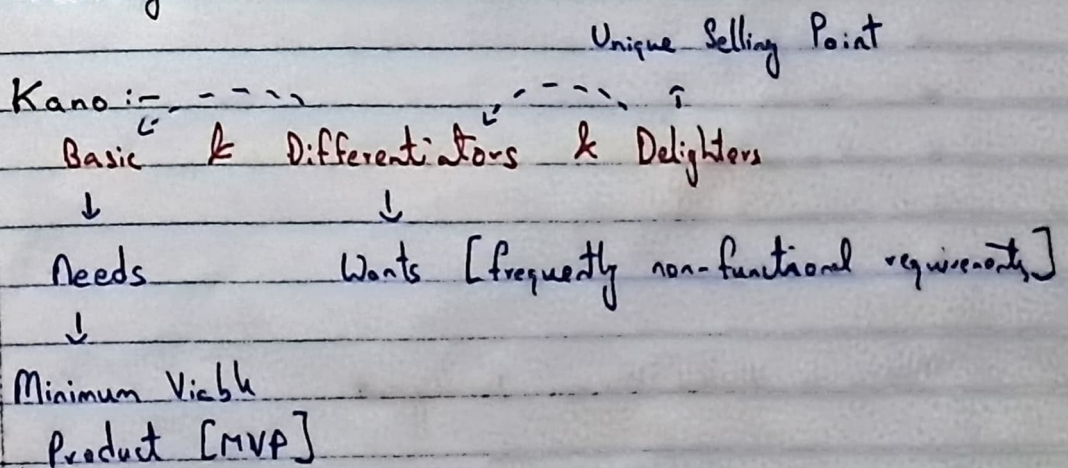
MVP :- Minimum Viable Requirements Product

MoSCoW method :- Must have, Should have, Could have, Won't have → Misuse cases opposite of user stories

Other methods

- (i) Interviews
- (ii) Observation
- (iii) Workshop / Joint Application Development

Quality Control :-



Good requirements engineering reduces rework.

Requirements Quality Control

Check if the identified requirements represent all expectations of the client

Validation :- Am I building the right product?

Verification :- Am I building the product right?

- ↳ Clearly understand the user req
- ↳ Ensure quality characteristics are met.

- * Correctness
- * Completeness
- * Unambiguity
- * Feasibility
- * Necessity
- * Priority
- * Consistency
- * Concise
- * Traceability
- * Standard Conformity
- * Verifiability

MDD ← Model Driven Development

1 SRS document

One way of specification engineering

Could use propositional logic, but we end up using English

Our main requirements are functional

Non-fn requirements are tied directly or indirectly to fn reqs

- ↳ system level reqs
- ↳ tied to multiple fn reqs

Transitioning from Problem to Solution

Wants / Needs → what must the system do? → How should the system do it?

Requirements Modelling :-

Transition from textual requirements → visual design

Helps in validating requirements.

(i) Use case Case Modelling:-

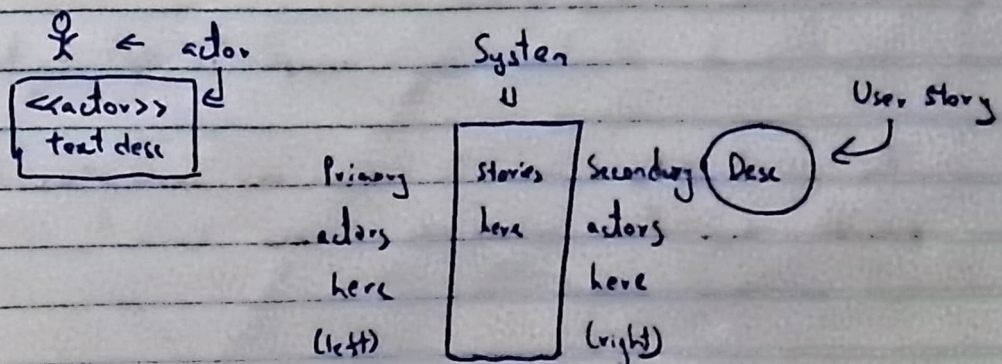
↳ a seq of actions including variants that a system or any other entity can perform, interacting with the actors of the system

Focuses on what to do not how to do it.

Actor:- Initiates specific behaviours of the system. Role played by entities interacting with the system.

- ↳ system, subsystems, classes

System/Subject Boundary:- Boundary b/w subject & actors



- (i) Identify actors $\leftarrow$ External & internal
- (ii) Identify goals $\leftarrow$ What does the user want to achieve
- (iii) Define use cases $\leftarrow$ One per goal.

There is a use case template

Post conditions $\leftarrow$ Implementation of behaviour

Scenario:- Specific seq or path in a use case

Use case instance:- execution of scenario

Per system design.

We must now define the use cases in detail

Needs to contain certain basic information.

Use case Name

Description

Actors

Pre-conditions

Main flow

Alt. flow

Post conditions

Conceptual Model:-

Identify & structure key concepts in the system's domain

The N & NPs in the Use Case description list the possible candidate conceptual classes

Not software classes

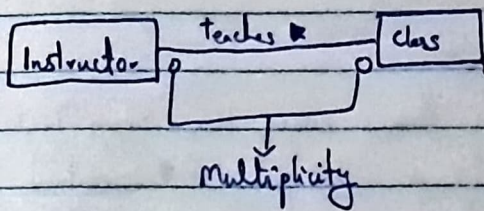
^{primitive}
 actual state of specific
 candidate class $\uparrow$
 Identify attributes & relationships b/w classes
 Validate into for completeness & realism
 Association; direction reading arrow
 $\rightarrow$ no arrow means direction of reading the label

Private $\Leftarrow -$

Public $\Leftarrow +$

Protected $\Leftarrow \#$

$[a..b] \Leftarrow$ at least a , at most b occurrences



UML $\Leftarrow$ Unified Modelling Language

Testing - Guest Lecture

Test:- Activity in which a system or a component is executed under specified conditions, the results are observed & recorded & an evaluation is made of some aspect of the system

Test Case:- A set of preconditions, inputs & expected results, developed to drive the execution of the system.

Exhaustive Testing:- Test every possible input to every component of your system.

Testing based on equivalence classes:-

Class Group of like objects

Equivalence Classes:- Classes that are (expected to be) treated equally by the system

If we assume that every input is an equivalence class is treated the same, then testing one input in the group is sufficient

But this fails in case of inputs with limited scope (boolean, etc.)

We also need to test combinations, boundaries, 0 values [edge cases] and also perform some random tests.

Based on coverage:-

- 1) Statement coverage (has every line been tested once)
- 2) Condition coverage (have all paths of conditions been covered)
- 3) Decision coverage (more general than condition)
- 4) Branch coverage (Decision + implicit conditions)
- 5) Modified Condition Decision Coverage (Every input is toggled independently)

Form Testing:-

Testing should always gather evidence, and testing should not be done by the same team as the developer

Why Test?

↳ Defects

↳ Evidence for conformance to

↳ Specs

↳ Stds

↳ Logs

↳ Safe & Unsafe scenarios to use a product

↳ Assess the quality

↳ Support for decision making

Types of testing:-

- ↳ Unit
- ↳ Integration
- ↳ System
- ↳ Acceptance

Black box & White box

↓

→

You only know
the requirements to be
met

You know what to test exactly
All levels supported

For system level

Design Level

A

B

C

D

↓

↓

↓

Fail only
once in
1 bil.

Statement

All elements coverage is sufficient
on a phase

∃ a format to write a test case.

Contains

- ↳ a unique id #
- ↳ objective
- ↳ priority
- ↳ traceability
- ↳ pre condition #
- ↳ inputs #
- ↳ expected outputs #

Test procedure:- Multiple sets of test cases.

Collection of test cases that need to be executed in a particular

order.

Mutation Testing:- To check for bugs in the very design of the system

Unit:- That which we choose to not divide further

Assigning random constants to variables not being tested.

Testing:- involves testing in isolation, diff test env than the prod env, fast regression, stubbing, testcases only target a unit. The coverage is local. Benefits :- Changes become easy to handle, integration becomes simpler, good documentation, helps in clarifying specs and any improvements.

System:- Collection of units.

Local Coverage:- Inputs can be manipulated to get 100% coverage of a unit, but doesn't work when the unit is a system.

Global Coverage:- Testing of the units ~~and~~ integration within a system

Effective

Writing Unit Tests

- ↳ Understanding Requirements
- ↳ Complex Logic
- ↳ Dependencies (Mocking / Stubbing)
- ↳ State Management
- ↳ Test Maintainability
- ↳ Performance Over head
- ↳ False Positives / Negatives
- ↳ Test Coverage
- ↳ Testing Edge Cases
- ↳ Maintaining Test Independence

Pack my box with five dozen liquor jugs

apsara

Date: _____

Integration Testing:- It's like checking correct data type rather than correct data. When more than one unit is tested to verify it's ^{correctly} interacting with each other. ^{data is being transferred}

- (i) Big-Bang → Integrate all at once.
- (ii) Top-Down → Integrate stubs first, then finalities
- (iii) Bottom-Up → Integrate subsystem by subsystem. Heilly
- (iv) Critical first → Single Point of Failure
- (v) Function at a time → All units available do fn by fn
- (vi) As-Delivered → As & when a unit is delivered
- (vii) Sandwich → Mix of (ii) & (iii)

System Testing:- Final system is tested to check compliance with the fn requirements and a non-fn requirements. Uses final real env, tests as both user and designer, checks performance - speed, latency, load, and tests how failures are handled. Last pt where ~~failur~~ failures are accepted.

Acceptance Testing:- The customer & end user tests the system. Beta, Crowd, Compliance & Field Testing.

Validates:- Business reqs, user workflows, real-world usage, contract conditions, regulatory rules.

Decide whether or not to release
Executed in a non-developmental perspective

Design:-
Good Design = reduced bugs.

Do NOT REINVENT THE WHEEL

Arch = High level design.

We should try to incorporate ~~that~~ which exists already in our system rather than rebuilding it. $\rightarrow$ How do objects talk to each other.

Class Diagram $\downarrow$ Structure & Dynamics
 Seq Diagrams and State Diagrams $\leftarrow$ inter-object behaviour
 $\leftarrow$ intra-object behaviour
 $\leftarrow$ representation of objects as state machines
 $\leftarrow$ Behaviours & Interaction

Components, Constraints & Connections $\in$ CCC

Design representation = {Seq Diags, State Diags, State Machines}

Design Aspects:-

- $\hookrightarrow$ Architecture Design _{subsystems and}
 - $\hookrightarrow$ Division into components $\leftarrow$ their connection, interaction & interfaces
- $\hookrightarrow$ Class Design
 - $\hookrightarrow$ Various features of class
- $\hookrightarrow$ User interface design
- $\hookrightarrow$ Algorithm design
 - $\hookrightarrow$ Design of computational mechanisms
- $\hookrightarrow$ Protocol design
 - $\hookrightarrow$ Design of communications protocol.

Main component of design is decision making

What is Software Design?

Expresses a soln to a problem in terms that are programming language agnostic.

Designs should be implementable in any language.

Fundamental Concepts:-

Abstraction $\Leftarrow$ Data, procedure, control $\Leftarrow$ ^{To abstract information} Not to that is not necessary for others
Separation of Concerns $\Leftarrow$ any complex problem is more easily handled if it is subdivided into pieces

Procedural Abstraction $\Leftarrow$ Considering the system to be a black box, code base is rarely available.

Control Abstraction $\Leftarrow$ Not every actor needs to be able to control everything. Exception Handling, iterators, subroutines

SoC (or) Decomposition $\Leftarrow$ Concern is a feature or behaviour that is specified as a part of the requirements model for the software.
 No notion of centrality. Monolithic software is bad since it is harder for engineers to understand.

But how fine grained is too fine grained?

Minima of the lines of code per module vs Cost of Integration of modules
 Less modules = Less cohesion

Information Hiding $\Leftarrow$ Methods can be public, but data should be private.
 Share info only when it's necessary!

Stepwise Refinement *

Coupling $\therefore$ More modules $\Rightarrow$ dependencies b/w modules.

Good Design $\rightarrow$ Loose Coupling + High Cohesion.

Blob

Design Smells!

S.O.L.I.D Principles

Single $\rightarrow$ Open/Closed $\rightarrow$ L's

Responsibilities Principle

Very Nice

- S $\in$ Single Responsibility
- O $\in$ Open / Closed Principle
- L $\in$ Liskov Substitution $\in$ Subclasses must be substitutable for their base
- I $\in$ Interface Segregation $\in$ No client should depend on methods it doesn't use
- D $\in$ Dependency inversion $\in$ Depend on abstractions not concretions.

Secure By Design:-

Settlement:- Settling all the ledger entries at the end of a time period (usually 4 hours, 1 day, 1 month etc)

Salami Attack:- Converting b/w currencies had a loss of \$1 for every \$1,000,000, redirecting this to a different account to profit.

Rue - Cry - Pry = Rules, Criteria, Priority

Non fn Testing:- Performance, Privacy, Recovery/Resilience

SQL Injection, Cross-site scripting, Man in the middle attack
Data Violation, nuisance generation

NEFT = 3hrs

RTGS = 1-3s (Used by UPI)

SFMS is the base for all of this

User Interface Design & Usability Engineering

Non Fn Reqs

To compare different designs, we use the NFRs as the metric

There is always a trade-off b/w the different NFR :

UI vs UX (User Experience)

UX contains UI

UI are the elements themselves

UX is the ability to easily and properly to these elements

USABILITY

Tradeoff $\in$ Security vs Accessibility.

Should we have to use VPN to log into IMS?

Who are the kinds of people expected to use your products?

We need to know the Users' needs, expectations, behaviours & goals.

We need to define personas

We need to think of the data flow through the system

UI $\rightarrow$ colour, content, branding, layout, visual $\Leftarrow$ Bells & Whistles

UX $\rightarrow$ Nav, Analysis, Usability, Interaction $\Leftarrow$ contains UI

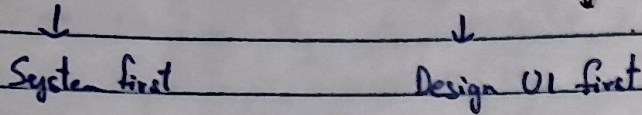
The closer the actual model is to the expected mental model, it is considered to be better.

The model/system should be easy to learn

Bridge the gap b/w the mental model & the actual working of the system

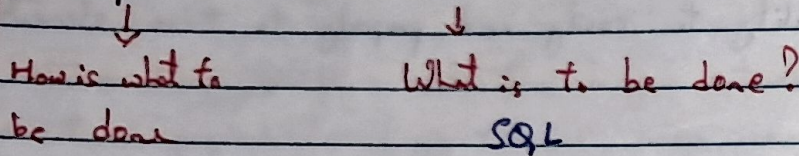
Mental Model:-

Inside Out vs Outside In Design:



Imperative vs Declarative

⚡



C, C++

Python

Most programming languages

SQL



Most Markup / Design Languages

"What" is provided through Class Diagrams

"How" is provided through Seq & State Diagrams

Seq Diagram => seq of method calls to accomplish a use case

Intra object behaviour = decompose objects into subobjects

Model = Description of something. Has abstractions

Models should be consistent

Model Maturity Levels:-

Lvl-0 = No spec

Lvl-1 = Textual

Lvl-2 = Text with diagrams

Lol-3 = Models with text.

Lol-4 = Precise models

(Modelling)

Domain Specific Languages

Languages designed to model a certain domain.

General Purpose Modelling Languages

Languages that can be used applied to any domain for modelling.

Push for models as first class citizens rather than code

Modelling languages have a constrained vocabulary → easier to describe objects

OCL = Object Constrained Language → GPM, subset of UML

UML models processes, it is not a process

BPML ← Business Processes Modelling Language

Models & Meta Models

↳ Model of Models

Objects are instances. They are realisations of models, which themselves are realisations of meta-models

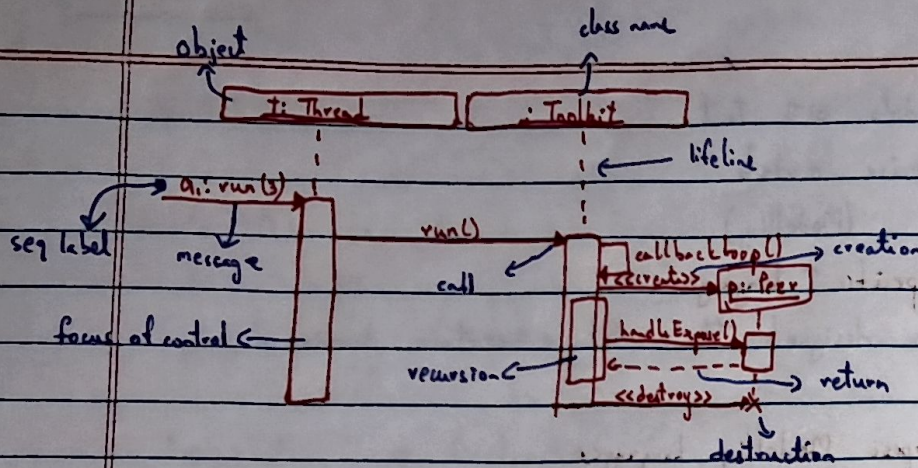
Sequence Diagram:

Captures Dynamic Behaviour

Purpose:-

↳ Model flow of control

↳ Illustrate typical scenario.



Systems are created by supplying or consuming services.

Service $\leftarrow$ any code that achieves some task

web services $\leftarrow$ services hosted on the web

microservices $\leftarrow$ large services decomposed.

Modern day systems are comprised of many micro-services.

System Seq Diagram $\leftarrow$ from use case

Objects are class names in class diagrams.

Interaction sequence is always $L \rightarrow R$; $top \rightarrow down$

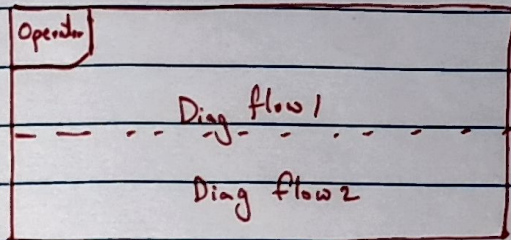
Peer is placed below Thread & Toolkit since it does not exist before the sequence is executed.

: is the life line. It says that the object exists and ceases to exist upon destruction [X]. This is the essential activation of an object.

█ shows the activation of the object (say a thread that is running).

alt in a box is used to represent alternate flows.
par in a box is used to represent parallel flows.

Many such modifiers exist.



Guards:- Allow for conditional transitions

Transition:- Relationship b/w two objects states indicating that an object will move from one state to the other

Class, Seq & State Diagrams